

## MP1

In this Mini-Project, you will create a website based on your most-recent **resume**.

### Task-1:

In this project none of your HTML files and none of your folders should contain spaces and capital letters, (**DO NOT** capitalize and **NO** spaces).

- Name the project **mp-1** but your home-page must be “**index.html**”, and start with the home-page of your resume.

### Requirements:

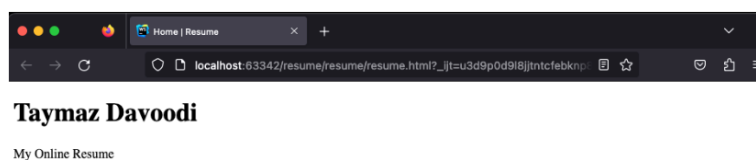
- Your online resume **must include at least 6 internal webpages**, (based on sections of your resume). For example, an internal page about your education, another about your work experiences, another one about your certifications, etc.
- One of the internal webpages **must** be **projects.html**. You **must** have this internal page, even if you do not have a **Projects** section in your resume. If you do have a Projects section in your resume, you may add your content to this page but please leave some space at the bottom because we will add a **JavaScript Calculator** at the bottom of this internal page, which should serve as a JS project.

## HTML

All internal webpages, and your website’s **Home** page, **must** have the 4 main tags (**header**, **nav**, **main**, and **footer**):

The **<header></header>**:

- Should include a title (**<h1>**) for the webpage and subtitle (**<p>**) describing it.



The **<nav></nav>**:

- Must include at least 6 links (**<a>**) to internal webpages, but may be more than 6, (e.g., Home, Education, Experiences, Certifications, References, Achievements, Projects, etc.).

- Each link must have an “href” attribute:
- Remember that those links would have to be nested inside <li> tags, and <li> tags must be nested inside a <ul> tag.

- [Home](#)
- [Education](#)
- [Employment](#)
- [Achievements](#)
- [References](#)
- [Documents](#)

### The <main></main>:

- Each internal page (including [Home](#)) must have a unique content based on one of the sections of your resume. The content must occupy at least half of the screen.

#### Home



My name is Taymaz Davoodi, and I am a part-time Computer Science lecturer at Boston University. I would like to further my academic and professional involvement in teaching, research, or administrative capacity, while I continue to advance towards a PhD or a full-time teaching position

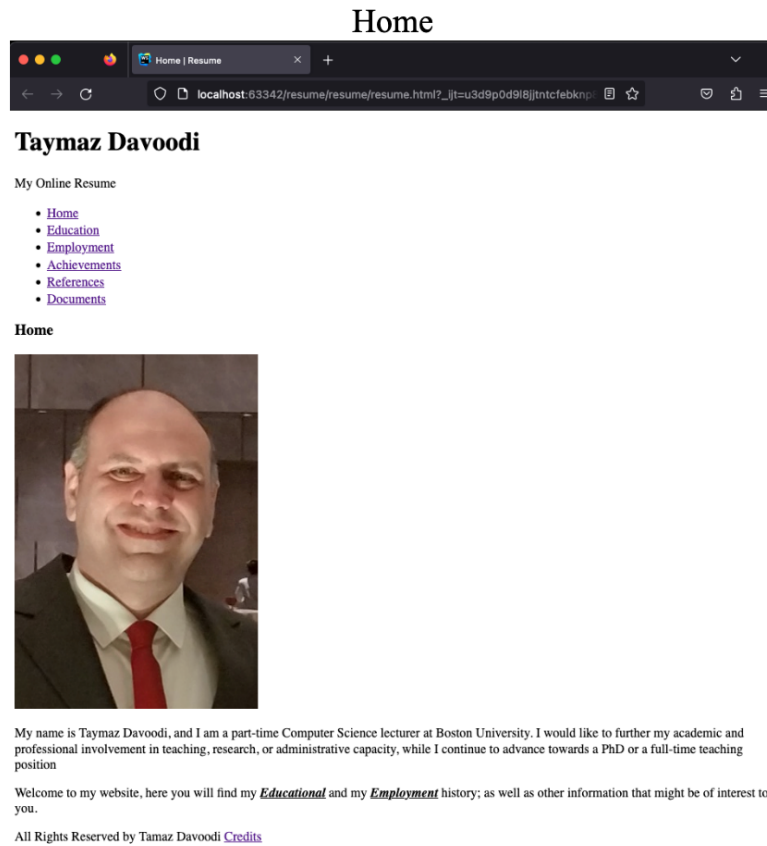
Welcome to my website, here you will find my [Educational](#) and my [Employment](#) history; as well as other information that might be of interest to you.

### The <footer></footer>:

- Should include a **copyrights statement** (<p>).
- The **copyrights statement** should include a link (<a>) named [Credits](#) and a **copyrights logo** (&#169; or &copy;).

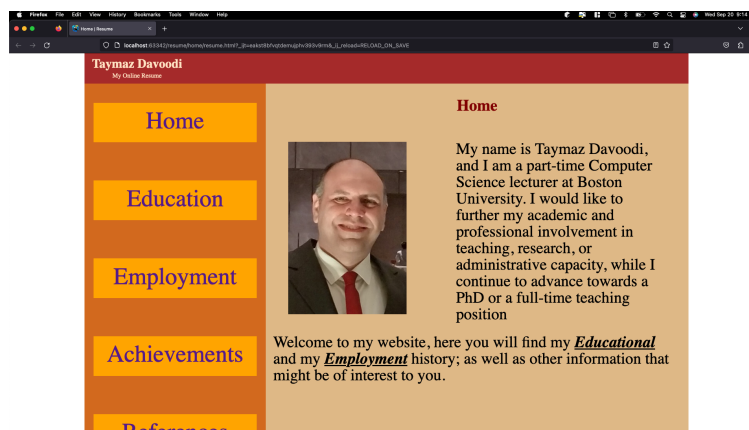
All Rights Reserved by Tamaz Davoodi [Credits](#) ©

Ex:



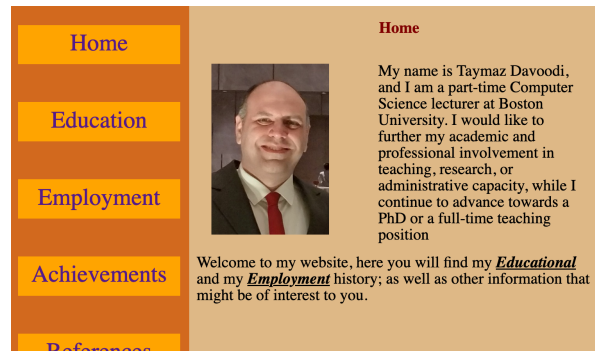
## CSS

All internal webpages, including the Home page, **must be responsive, wrapped**, squeezed with **margins on both sides, and centered**:

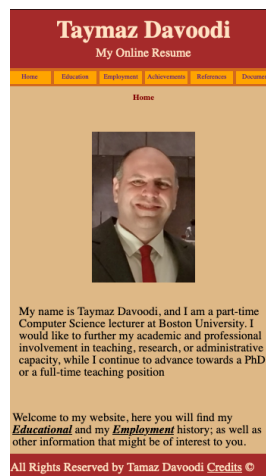


## The nav{}:

- For screens bigger than 750-px: should not exceed 30% of the width of its parent, and must be vertical.



- For screens smaller than 750-px: should occupy 100% of the width of its parent and appear horizontal.



## The header{}:

- For screens bigger than 750-px: the content should be aligned left.

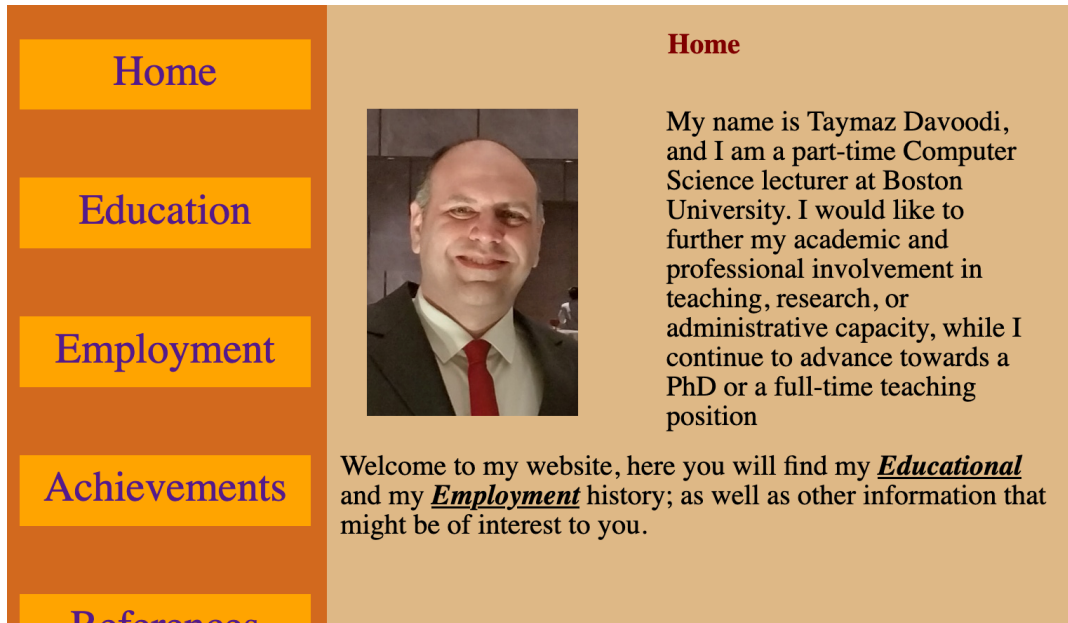


- For screens smaller than 750-px: the content should be aligned center.

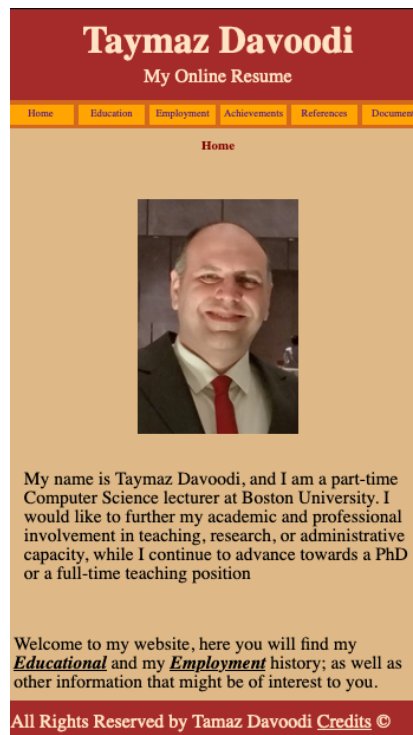


The main{}:

- For screens bigger than 1000-px: should not exceed 70% of the width of its parent.

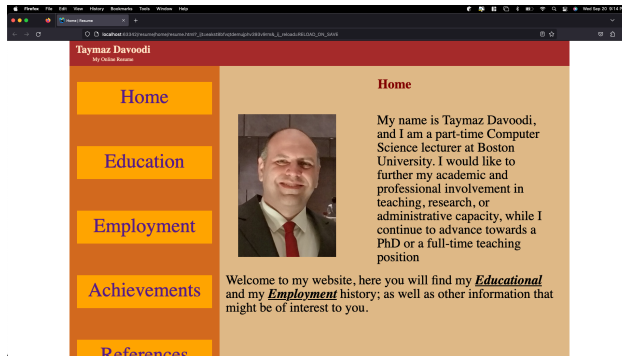


- For screens smaller than 1000-px: should occupy 100% of the width of its parent

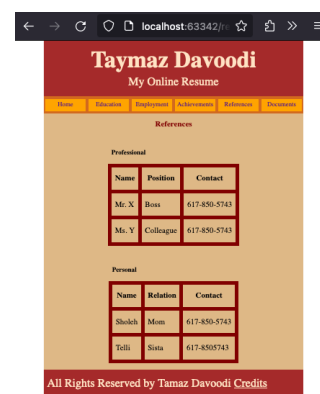
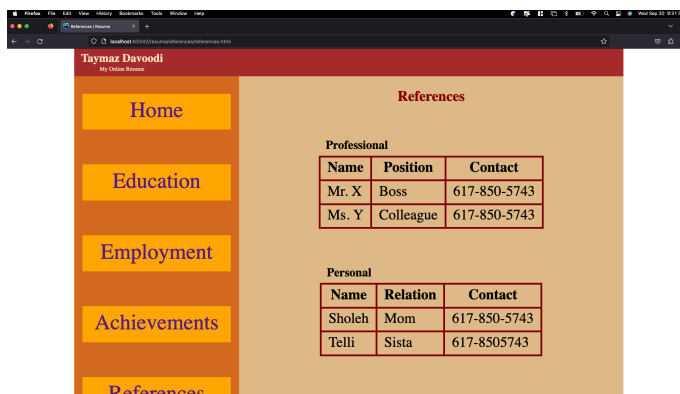


- Example:

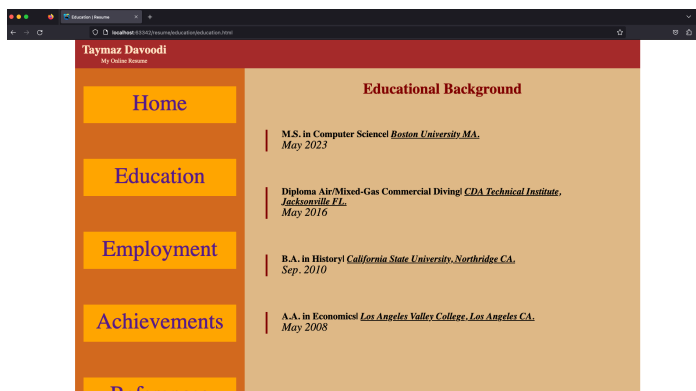
## Home



## References



## Education



Etc.

**Note:**

- Make good use of **margins** and **padding**s. Leave open spaces and do not clutter. You are being graded for technical as well as aesthetic aspects.
- Margins to the left and right of the screen is **expected**, but there should **not** be any white spaces at the bottom.  
hint: You should increase the height of your `<main>` to **100vh**.
- Use **calc( + )** for font-sizes.
- Chose a background color for your main tags (**header, nav, main, footer**), but maintain a sense of uniformity.
- Use customized fonts, like google fonts.
- You may use Flex-box with Media Queries.

## JS

In your internal **Projects** webpages, we are going to add a **JavaScript Calculator**. Remember that this page is an internal page and just like all the other internal pages must have the 4 main tags (header, nav, main, and footer). If you do have worked on other projects, and have shared them on GitHub you may post links to those repositories in your Projects webpage, but a some space at the bottom of the page must be reserved for our calculator.

**Our calculator should include:**

- 2 **input** boxes.
- 6 **buttons** marked (+, -, \*, /, \*\*, Clear).
- 1 **output** textual tag (e.g., `<p></p>`, `<h1></h1>...<h6></h6>`).

The user should be able to enters a number into the first and the second **input** box, and click on one of the **buttons**, to see the result of the operation displayed in the **output** box.

- The **input** tag specifies an input field where the user can enter data. It is a stand-alone element. This element contains only attributes and it does not have any content.

Ex:

```
<input type="text" id="first-number">
```

- The **button** tag defines a clickable event. Inside a button element you can put content, like text or images.

Ex:

```
<button onclick="addition()"> + </button>
```

Note:

For the onclick= " " attribute of your addition button, **DO NOT** write "add()".  
add() is a pre-existing function in Js. and we don't want to use that one.

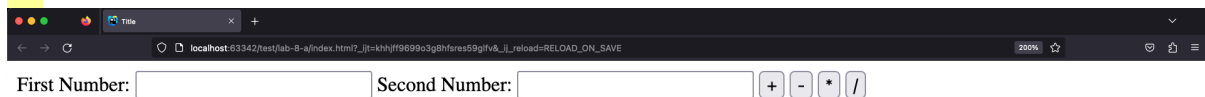
- The **output** should be displayed in textual tag using the **.innerHTML()** function.

Ex:

```
.html
<h3 id="output"></h3>
```

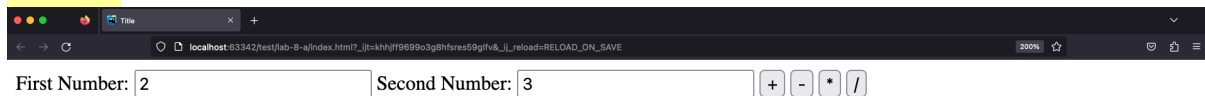
```
.js
document.getElementById("output").innerHTML=String(result);
```

Ex:



First Number:  Second Number:  + - \* /

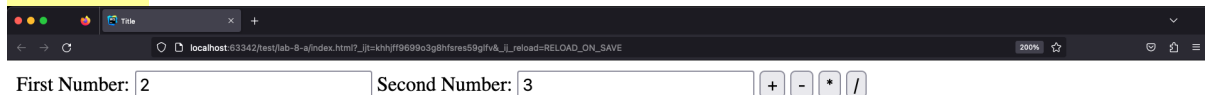
Addition:



First Number: 2 Second Number: 3 + - \* /

5

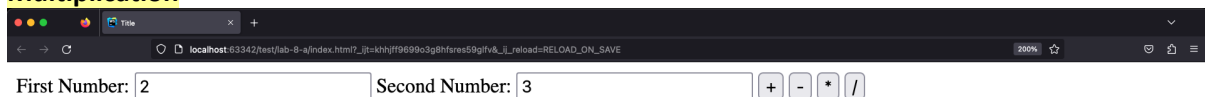
Subtraction



First Number: 2 Second Number: 3 + - \* /

-1

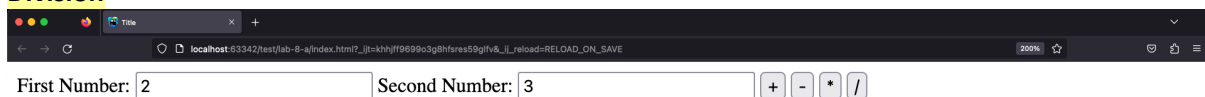
Multiplication



First Number: 2 Second Number: 3 + - \* /

6

Division



First Number: 2 Second Number: 3 + - \* /

0.6666666666666666

- In your **.html** file:
  1. Add a **power** button; when it is pushed it should raise the value of the first input to the value of the second input.

2. Add a **clear** button; when it is pushed, it should erase the contents of the 2 input boxes and the output.
- In your **.js** file:
    1. For the **power()** function you must use a **for()** loop. **If you implement your power() function using anything but a for() loop, (like Math.pow() or \*\*), you will get a 1/3 deduction on MP1's grade.**
    2. For the **clear()** function you can assign an empty **""** to the value of the input boxes and the **.innerHTML** of the output tag.
    3. For your results, regardless of which button is pushed, If the value of the result is negative, the result should be displayed with either **red** text or a **red background**.



## Style your calculator!!

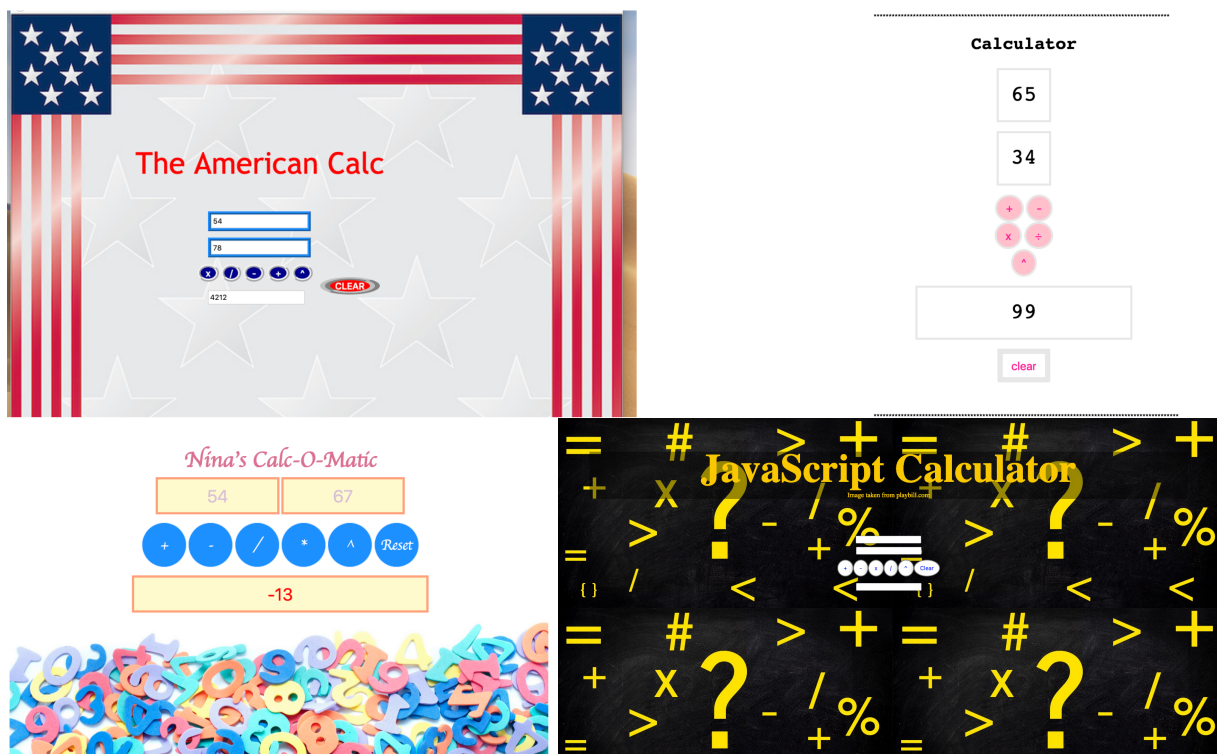
- At this point you should have a working multifunction calculator, but it might not be the prettiest thing on the web.
- In this part you'll apply some creative CSS to snap things up a bit.

This means that you should:

1. Use **div** elements to group and align the inputs, buttons, result, and for any styling that you want.
2. Make good use of margins and paddings. Leave open spaces and do not clutter.
3. Use **calc( + )** for font-sizes.
4. Use customized fonts, like google fonts.
5. Use **Flex-box with Media Queries**.

6. Make sure the content doesn't exceed the limitations of the screen or their parent tag, even when the screen is at smallest size (mobile device).
7. Make sure the text remains readable even if the screen is at the smallest size (mobile device).
8. Make sure margins and paddings shrink when the screen gets smaller.
9. Make sure you apply max-width for any image, so that it becomes responsive to the screen size.

The following images are some examples of calculators (from past semesters):



## Task-2:

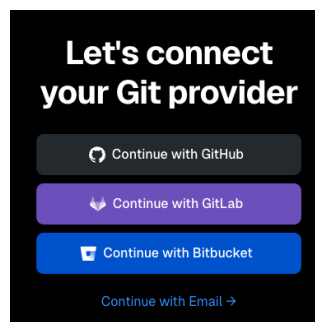
### **Deployment:**

1. In your terminal, navigate to the root of the directory of your project (mp-1) and use the command **git init**
2. Use the command **git status** to see the current status of the files within your directory. Right now, all files should be untracked
3. Use the command **git add filename** or **git add .** (to add all files). This tells Git about any changes you've made to this file since its last commit.

4. Once you have added all relevant files, use the command **git commit -m "your commit message"** to commit your changes. This confirms your staged changes with Git. Be sure to include a helpful/informative commit message when using this command.
5. Create a new GitHub repository by visiting <https://github.com> and clicking **New** on the left next to **Top Repositories**
6. Give your repository the same name as the directory you're working in (**mp-1**) and click **Create repository**
7. Follow the directions under **or push an existing repository from the command line** in your terminal
8. You may be prompted by Git to use the command **git push --set-upstream origin main**. If you are, do so.
9. Your files should now be on GitHub! Refresh your GitHub tab if you don't see your files.
10. Once you saw your files, **Save the URL**:  
(e.g., [https://github.com/YOUR\\_USER\\_NAME/mp-1](https://github.com/YOUR_USER_NAME/mp-1)).

#### Hosting:

1. Visit <https://vercel.com/signup> and set up a free (Hobby) account (you might have done this already in **Dis-1**).
2. Connect your GitHub account to your Vercel account.



3. Once connected all of your GitHub repositories will be listed in Vercel. Select **mp-1** from the list and push the **import** button next to it.
4. In the next screen you don't need to change anything; just click **Deploy**. Vercel will take a few seconds to process the files and will prompt you with a "Congratulations" message and a screenshot of your hosted app.
5. Click on the screenshot, which should open a new browser-tab, **save its URL**:  
(e.g., <https://mp-1.vercel.app>).

**Submission:**

- Create simple **text file** with any writing platform (MS-word, notepad, etc.)
- Write your first-name, last-name, and BU-username on top.
- One the next line paste **both saved URLs**:

(e.g., [https://github.com/YOUR\\_USER\\_NAME/mp-1](https://github.com/YOUR_USER_NAME/mp-1)).

(e.g., <https://mp-1.vercel.app>).

- **You Link might be different from the ones above, so make sure that both open to your assignment before you submit them.**
- Add comments if you like.
- Convert it to **PDF** as **mp-1.pdf**.
- Navigate to [gradescope](https://gradescope.com), find CS-391, find this assignment, and submit it as: **(mp-1.pdf)**.

**Note:**

**DON'T UPDATE/PUSH/change** your assignment's GitHub repo after the **due-date**. **Altering your assignment after its due-date is considered cheating** and at minimum, will result in an automatic 0 on that assignment.

You may change or alter that repository after you received a grade on that assignment.